COMSATS University Islamabad

Attock Campus

Department Of Computer Science

Course	***Information Security***
Instructor	***MAM AMBREEN GUL***
Assignment No.	***03***

Student Details

Registration No.	***Name***
FA24-BSE-058	***Adeel Abbas.***

Task 1: Basic Blockchain Implementation

Code:

```python
import hashlib
import datetime

# ------------------------------
# Block Class
# Represents a single block in the blockchain
# ------------------------------
class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index                      # Position of block in chain
        self.timestamp = datetime.datetime.now() # Time of creation
        self.data = data                        # Transaction or stored data
        self.previous_hash = previous_hash      # Hash of previous block
        self.hash = self.generate_hash()        # Current block hash

    # Function to generate SHA256 hash of the block
    def generate_hash(self):
        # Combine all attributes into a single string
        content = str(self.index) + str(self.timestamp) + str(self.data) + str(self.previous_hash)

        # Encode and hash using SHA256
        return hashlib.sha256(content.encode()).hexdigest()


# ------------------------------
# Blockchain Class
# Manages the chain of blocks
# ------------------------------
class SimpleBlockchain:
    def __init__(self):
        self.chain = []                 # List to store blocks
        self.create_first_block()       # Create genesis block

    # Create the first block (no previous block exists)
    def create_first_block(self):
        genesis_block = Block(0, "Genesis Block", "0")
        self.chain.append(genesis_block)
```

```python
    # Add a new block to the chain
    def add_block(self, data):
        last_block = self.chain[-1]     # Get the last block in chain

        # Create new block using last block's hash
        new_block = Block(len(self.chain), data, last_block.hash)

        self.chain.append(new_block)

    # Display all blocks in the chain
    def show_chain(self):
        for block in self.chain:
            print(f"\nBlock {block.index}")
            print("Timestamp     :", block.timestamp)
            print("Data          :", block.data)
            print("Previous Hash :", block.previous_hash)
            print("Hash          :", block.hash)

    # Verify integrity of blockchain
    def verify_chain(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i - 1]

            # Check if link between blocks is broken
            if current.previous_hash != previous.hash:
                return False

        return True


# ------------------------------
# Testing the Blockchain
# ------------------------------
if __name__ == "__main__":
    blockchain = SimpleBlockchain()

    # Adding blocks with sample data
    blockchain.add_block("A sends 10 coins to B")
    blockchain.add_block("B sends 5 coins to C")
    blockchain.add_block("C sends 2 coins to D")
    blockchain.add_block("D sends 1 coin to E")

    print("=== Original Blockchain ===")
    blockchain.show_chain()

    print("\nIs blockchain valid?", blockchain.verify_chain())

    # ------------------------------
    # Tampering demonstration
    # ------------------------------
    print("\n=== After Tampering ===")

    # Modify data of a block
    blockchain.chain[2].data = "HACKED DATA"

    # Recalculate hash of tampered block
    blockchain.chain[2].hash = blockchain.chain[2].generate_hash()

    blockchain.show_chain()

    # Validation should now fail
    print("\nIs blockchain valid?", blockchain.verify_chain())
```

OUTPUT:

```
Block 0
Timestamp     : 2026-05-02 20:42:56.881232
Data          : Genesis Block
Previous Hash : 0
Hash          : 8e9340a1f7711504e9e8d03ee7caba997bf0869d4971e17d15a42f134b01e0f7

Block 1
Timestamp     : 2026-05-02 20:42:56.881305
Data          : A sends 10 coins to B
Previous Hash : 8e9340a1f7711504e9e8d03ee7caba997bf0869d4971e17d15a42f134b01e0f7
Hash          : 8c83a11a41593251333d2d378a5b9b4cf721d95ea6f34b40e89e064effc75c31

Block 2
Timestamp     : 2026-05-02 20:42:56.881323
Data          : HACKED DATA
Previous Hash : 8c83a11a41593251333d2d378a5b9b4cf721d95ea6f34b40e89e064effc75c31
Hash          : b65e698530fba25a7954e16d4a55e35d2ccbb1d7e171778d90b99de3b2a36257

Block 3
Timestamp     : 2026-05-02 20:42:56.881329
Data          : C sends 2 coins to D
Previous Hash : 9d88628b8aa17a5b6dcd9b78ddcf7326939b28b216c980f2399a41fde69a3164
Hash          : 40f0e8aca57de86a67b5d139a7e2f9d46c320c487c8f605ff0d563bdd1c5d624

Block 4
Timestamp     : 2026-05-02 20:42:56.881334
Data          : D sends 1 coin to E
Previous Hash : 40f0e8aca57de86a67b5d139a7e2f9d46c320c487c8f605ff0d563bdd1c5d624
Hash          : 04b857898deee472d7858bbb70e2f5c3d9bae1a3d674017993d73e60948afa11

Is blockchain valid? False
PS C:\Users\Timeline\Desktop>
```

Key Concept

- Each block stores the **hash of the previous block**
- If one block changes → its hash changes
- That breaks the link → chain becomes invalid

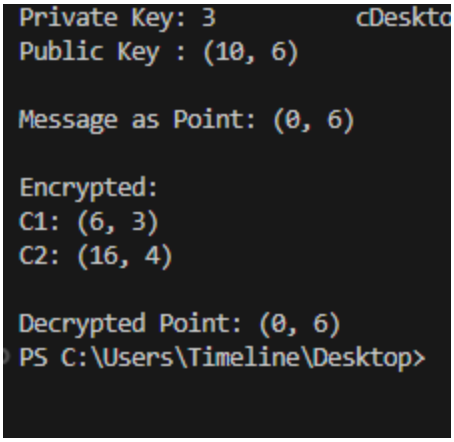
Task 2: Elliptic Curve Cryptography (ECC) Basics

Code:

```python
# ========================================
# Elliptic Curve: y^2 = x^3 + ax + b mod p
# Using small numbers for learning purposes
# ========================================

p = 17   # Prime modulus
a = 2
b = 2

# Base point (must satisfy curve equation)
G = (5, 1)


# ========================================
# Modular inverse using Python built-in pow
# ========================================
def mod_inverse(k, p):
    return pow(k, -1, p)


# ========================================
# Point Addition: P + Q
# ========================================
def point_add(P, Q):
    # If P == Q, use doubling formula
    if P == Q:
        return point_double(P)

    x1, y1 = P
    x2, y2 = Q

    # Calculate slope (m)
    m = ((y2 - y1) * mod_inverse(x2 - x1, p)) % p

    # Calculate resulting point
    x3 = (m*m - x1 - x2) % p
    y3 = (m*(x1 - x3) - y1) % p

    return (x3, y3)


# ========================================
# Point Doubling: 2P
# ========================================
def point_double(P):
    x, y = P

    # Calculate slope (m)
    m = ((3*x*x + a) * mod_inverse(2*y, p)) % p

    # Calculate resulting point
    x3 = (m*m - 2*x) % p
    y3 = (m*(x - x3) - y) % p

    return (x3, y3)


# ========================================
# Scalar Multiplication: k * P
# (Repeated addition)
# ========================================
def scalar_multiply(k, P):
    result = P

    for _ in range(k - 1):
        result = point_add(result, P)

    return result


# ========================================
# Inverse of a point (used in decryption)
# ========================================
def inverse_point(P):
    x, y = P
```

```python
    return (x, (-y) % p)


# ========================================
# MAIN PROGRAM (Testing)
# ========================================
if __name__ == "__main__":

    # ------------------------------
    # Key Generation
    # ------------------------------
    private_key = 3                      # chosen small integer
    public_key = scalar_multiply(private_key, G)

    print("Private Key:", private_key)
    print("Public Key :", public_key)

    # ------------------------------
    # Message Encoding
    # Convert number → point
    # ------------------------------
    message = 7
    M = scalar_multiply(message, G)

    print("\nMessage as Point:", M)

    # ------------------------------
    # Encryption (EC-ElGamal simplified)
    # ------------------------------
    k = 2   # random number

    C1 = scalar_multiply(k, G)
    C2 = point_add(M, scalar_multiply(k, public_key))

    print("\nEncrypted:")
    print("C1:", C1)
    print("C2:", C2)

# ------------------------------
# Decryption
# ------------------------------
temp = scalar_multiply(private_key, C1)

# Subtract by adding inverse
decrypted = point_add(C2, inverse_point(temp))

print("\nDecrypted Point:", decrypted)
```

Output:



1. ECC Concept

- Uses curve:
  y2 = x3 + ax + b mod p
- Security based on:
  Elliptic Curve Discrete Log Problem

2. Key Generation

- Private key = random integer
- Public key = k × G

3. Encryption Logic

- Message converted to point M
- Cipher:
  - C1 = kG
  - C2 = M + kPb

4. Decryption Logic

- Compute:
  - kPb = private_key × C1
- Recover:
  - M = C2 − kPb

5. Security Analysis

Good points to include:

- ECC provides **strong security with small keys**
- Hard problem: finding k from kG
- Your implementation is **not secure in real-world** because:
    - small prime p
    - no padding / encoding
    - simplified mapping